Course Project: Matrix Multiplication from CPU to Optimized CUDA

Project duration

4 weeks

Overview

Matrix multiplication is simple to define but rich in parallel-programming concepts. In this project, you will implement the same general matrix multiplication operation in four progressively optimized ways:

- Single-threaded CPU.
- Naive CUDA GPU.
- Shared-memory tiled CUDA GPU.
- Shared-memory tiled CUDA GPU with padding to reduce bank conflicts.

The goal is not only to produce a fast final kernel. You must explain what changes at each stage, verify every implementation against the same reference, and use measurements to connect the observed performance to parallelism, memory coalescing, data reuse, synchronization, and shared-memory bank behavior. NVIDIA's CUDA Best Practices Guide describes shared memory as a user-managed cache for eliminating redundant global-memory transfers, while the matrix multiplication performance guide explains how GPU GEMMs partition output matrices into tiles and step through the reduction dimension.

Learning objectives

By completing this project, you should be able to:

- Map a nested-loop algorithm onto CUDA threads, blocks, and grids.
- Distinguish parallelism from memory-efficiency improvements.
- Achieve coalesced global-memory accesses.
- Use shared memory and synchronization correctly.
- Explain reuse and arithmetic intensity in a tiled algorithm.
- Identify and reduce shared-memory bank conflicts with padding.
- Design fair CPU/GPU benchmarks and interpret profiler metrics.

Problem statement

Given row-major matrices:

- $A \in \mathbb{R}^{M \times K}$
- $B \in \mathbb{R}^{K \times N}$

compute:

$$C = AB,$$

where:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

All four implementations must compute the same operation on the same input data. The required scalar type is FP32.

Required implementations

1. Single-threaded CPU

Implement the conventional three-loop algorithm in C or C++. It must run with one host thread. Parallel CPU libraries, OpenMP, BLAS, compiler auto-parallelization, and GPU calls are not allowed in this version.

You may choose the loop order, but you must state it and explain its locality implications. Use this implementation as the correctness baseline and as one speedup reference.

2. Naive GPU

Implement a CUDA kernel in which each GPU thread computes one output element C_{ij}. Each thread reads the required row of A and column of B from global memory and performs the complete length-K dot product.

Requirements:

- Use a two-dimensional grid and two-dimensional thread blocks.
- Coalesce accesses where practical.
- Perform bounds checks for arbitrary M, N, and K.
- Do not use shared memory in this version.

This version should demonstrate the benefit of large-scale thread parallelism while exposing the cost of redundant global-memory reads.

3. Tiled GPU

Implement a CUDA kernel that divides C into square output tiles. Each thread block cooperatively loads one tile of A and one tile of B into shared memory, synchronizes, computes partial products, and iterates across the K dimension.

Requirements:

- Use shared-memory arrays with logical dimensions `TILE_SIZE x TILE_SIZE`.
- Each loaded input element should be reused by multiple threads.
- Synchronize before consuming a newly loaded tile and before overwriting it in the next iteration.
- Zero-fill out-of-range elements in partial edge tiles.
- Guard output stores for non-divisible dimensions.
- Evaluate at least two legal values of `TILE_SIZE`, such as 8, 16, or 32.

The CUDA Programming Guide's matrix-multiplication example illustrates this block-level decomposition, and the CUDA Best Practices Guide explains how cooperative loading avoids redundant global-memory transfers.

4. Tiled GPU with padding

Starting from your tiled kernel, pad at least one shared-memory dimension so its physical leading dimension differs from the logical tile width. A typical declaration is:

```
__shared__ float tile[TILE_SIZE][TILE_SIZE + 1];
```

The extra column is not matrix-value padding and must not change the numerical result. Its purpose is to alter shared-memory address mapping and reduce bank conflicts for an access pattern that you identify. The CuTe GEMM tutorial similarly shows padded shared-memory layouts used to avoid bank conflicts.

Requirements:

- Preserve the same global-memory tiling and arithmetic as version 3 as much as possible, so padding is the primary experimental variable.
- Use Nsight Compute shared-memory conflict metrics to compare versions 3 and 4.
- Explain which access pattern is affected and why padding changes bank mapping.
- Report honestly if padding does not improve a particular kernel/GPU configuration; analyze the reason rather than hiding the result.

Padding matrices to handle dimensions that are not multiples of the tile size is a separate concern. All GPU versions must already support arbitrary legal dimensions through bounds checks or zero filling.

Input

The executable reads a binary input file with the following layout:

```
int32 M
int32 K
int32 N
float32 A[M*K]    // row-major
float32 B[K*N]    // row-major
```

All integers and floating-point values use the host machine's native little-endian representation. A provided or submitted generator must create valid files from a fixed random seed.

Required limits:

- $1 \le M, N, K \le 16384$
- Input values are finite FP32 numbers in $[-1, 1]$.
- Hidden correctness tests may use rectangular and irregular dimensions.
- Test dimensions may not be multiples of block or tile sizes.

Your implementation may reject a case with a clear error if its matrices do not fit in available host or device memory.

Output

Write a binary output file containing exactly `M*N` FP32 values in row-major order, with no header.

The command-line interface must be:

```
./matmul --impl {cpu,naive,tiled,padded} --input INPUT.bin --output OUTPUT.bin [--tile TILE_SIZE]
```

The program must validate arguments and CUDA API return codes. Unsupported tile sizes or insufficient resources must produce a clear message and a nonzero exit status.

Correctness requirements

Every implementation must be checked against the single-threaded CPU result for manageable sizes. Include tests covering:

- Square, tall, and wide matrices.
- Dimensions smaller than a CUDA block.
- Dimensions exactly divisible by the tile size.
- Dimensions one less and one greater than a tile multiple.
- M, N, or K equal to 1.
- At least one irregular case such as $M = 1000$, $K = 513$, $N = 769$.

Report maximum absolute error, maximum relative error, and whether all values satisfy a mixed tolerance. A recommended starting point is:

```
abs_error <= 1e-4 * K + 1e-4 * abs(reference)
```

You may use a tighter or better-justified tolerance. Floating-point addition order may differ between implementations, so bitwise equality is not required. Tests must use fixed seeds and reject NaN or infinity.

Performance evaluation

Measure both kernel execution and end-to-end execution:

- Kernel-only time excludes allocation and host-device transfers.
- End-to-end GPU time includes allocation, transfers, kernel execution, and result transfer.
- CPU time covers only multiplication after inputs are in memory.

Use CUDA events for kernel timing and a monotonic host clock for CPU and end-to-end timing. Include warm-up runs. Report the median of at least 20 trials for GPU kernels and enough CPU trials to obtain stable results.

Required benchmark shapes:

Shape (M, K, N)	Purpose
(256, 256, 256)	Small square problem and launch-overhead effects
(1024, 1024, 1024)	Medium square problem
(4096, 4096, 4096)	Large square problem, if memory/runtime permits

Shape (M, K, N)	Purpose
(2048, 512, 4096)	Rectangular output
(1000, 513, 769)	Irregular dimensions and edge handling
One instructor-approved shape	Explore a hypothesis of your choice

For each valid shape, report:

- Runtime for all four implementations.
- Speedup relative to single-threaded CPU.
- Speedup from naive GPU to tiled GPU.
- Speedup or slowdown from tiled GPU to padded tiled GPU.
- Achieved throughput in GFLOP/s using $2MNK$ floating-point operations.
- Kernel-only and end-to-end GPU results in separate columns or plots.

Use cuBLAS SGEMM only as an external performance reference. Calling cuBLAS, CUTLASS, WMMA, or another matrix-multiplication library does not satisfy any required implementation.

Profile one large divisible shape and one irregular shape with Nsight Compute. At minimum, discuss:

- Global load/store throughput.
- Global-memory load efficiency or sector utilization.
- Shared-memory bank conflicts.
- Achieved occupancy.
- Compute throughput and memory throughput.
- The limiting resource or bottleneck for each GPU kernel.

State the CPU model, GPU model, compiler flags, CUDA version, GPU clocks/power policy if known, tile size, and benchmark commands.

Four-week plan

Week 1: CPU baseline, test infrastructure, and naive CUDA

- Define the file format and implement the input generator.
- Implement the single-threaded CPU version.
- Build output comparison and automated correctness tests.
- Implement GPU memory management, transfers, launch error checking, and cleanup.
- Implement and validate the one-thread-per-output naive CUDA kernel.
- Measure initial kernel-only and end-to-end timings.

Milestone: the CPU and naive GPU versions pass the correctness suite and have an initial performance table.

Week 2: Shared-memory tiling and initial profiling

- Implement cooperative tile loading, synchronization, and partial-tile handling.
- Compare at least two legal tile sizes.

- Profile global-memory behavior, data reuse, occupancy, and throughput.
- Use the first profiler results to make an initial optimization.

Milestone: the tiled version is correct and its first performance improvement is explained with evidence.

Week 3: Padding and iterative profile-optimize loop

- Implement shared-memory padding while preserving the tiled kernel's arithmetic.
- Compare bank-conflict metrics for tiled and padded kernels.
- Repeat the profile-optimize cycle across divisible and irregular shapes.
- Tune tile size, memory-access patterns, and resource usage.
- Record successful and unsuccessful experiments.

Milestone: present a second profiler-guided improvement and explain whether padding helps on the selected GPU.

Week 4: Final optimization, benchmarks, and analysis

- Run the complete correctness and benchmark matrix.
- Compare all four implementations and the external cuBLAS reference.
- Analyze performance trends, edge effects, and remaining gaps to cuBLAS.
- Finalize code, scripts, plots, profiler evidence, and report.

Milestone: all four versions and reported results are reproducible from a clean checkout.

Deliverables

Submit one repository or archive containing:

- `README.md` with environment, build, usage, and reproduction instructions.
- C/C++ and CUDA source files for all four versions.
- A build system such as Make or CMake.
- Input generator and output-comparison utility.
- Automated correctness tests.
- Benchmark scripts and raw results in CSV or JSON.
- Profiler command lines and selected metric exports.
- A final report of 5–8 pages in PDF or Markdown.

The report must include pseudocode for each version, thread/block mapping, memory-access diagrams, correctness results, timing methodology, performance plots, profiler evidence, and an explanation of every optimization. Include a short section on unsuccessful experiments and what you learned from them.

Optional extensions

- Register tiling so each thread computes multiple output elements.
- Vectorized global-memory loads.
- Double buffering or asynchronous copies.
- FP16/BF16 inputs with FP32 accumulation.
- Tensor Core implementation as a clearly separated fifth version.

- Autotuning tile and block shapes.
- Roofline analysis.
- Batched matrix multiplication.

Suggested references

- CUDA Programming Guide: writing SIMT kernels
- CUDA C++ Best Practices Guide
- CUDA matrix multiplication performance guide
- CUDA C++ Programming Guide matrix multiplication examples
- CuTe dense matrix-multiplication tutorial